KENNESAW STATE
UNIVERSITY

CS 4277
CNN Case Study

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

Content

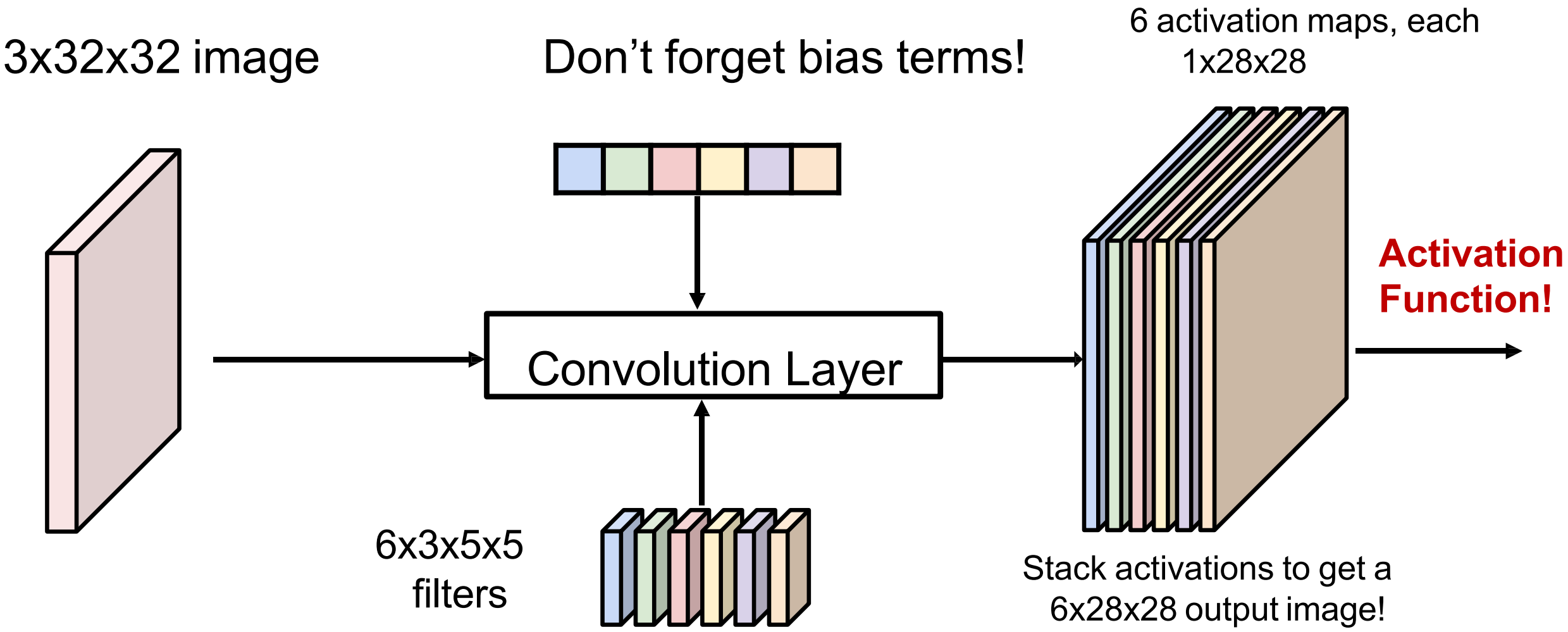
- Recap CNN
- Normalization
- CNN Architectures
- Weight Initialization
- CNN Training – Data Preprocessing & Augmentation
- Transfer Learning

Example of deep CNNs

Content

- Typical ConvNet
- Normalization
- How to build CNNs?
- How to train CNNs?

Recap: Convolution Layer



Recap: Max Pooling

Single depth slice

x

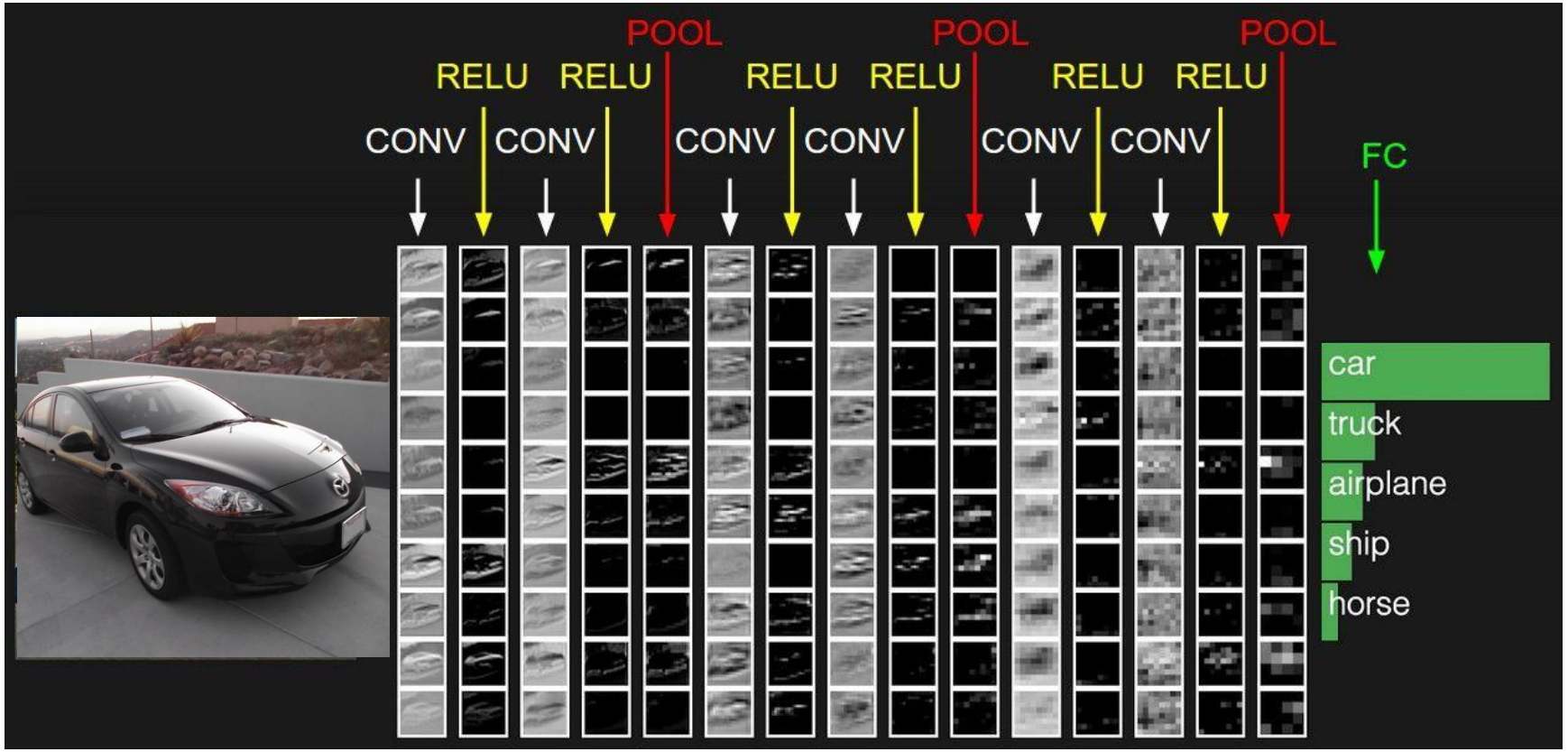
1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

y

max pool with 2x2 filters
and stride 2

6	8
3	4

Typical ConvNet



- Training

Softmax+CrossEntropy loss

- Testing

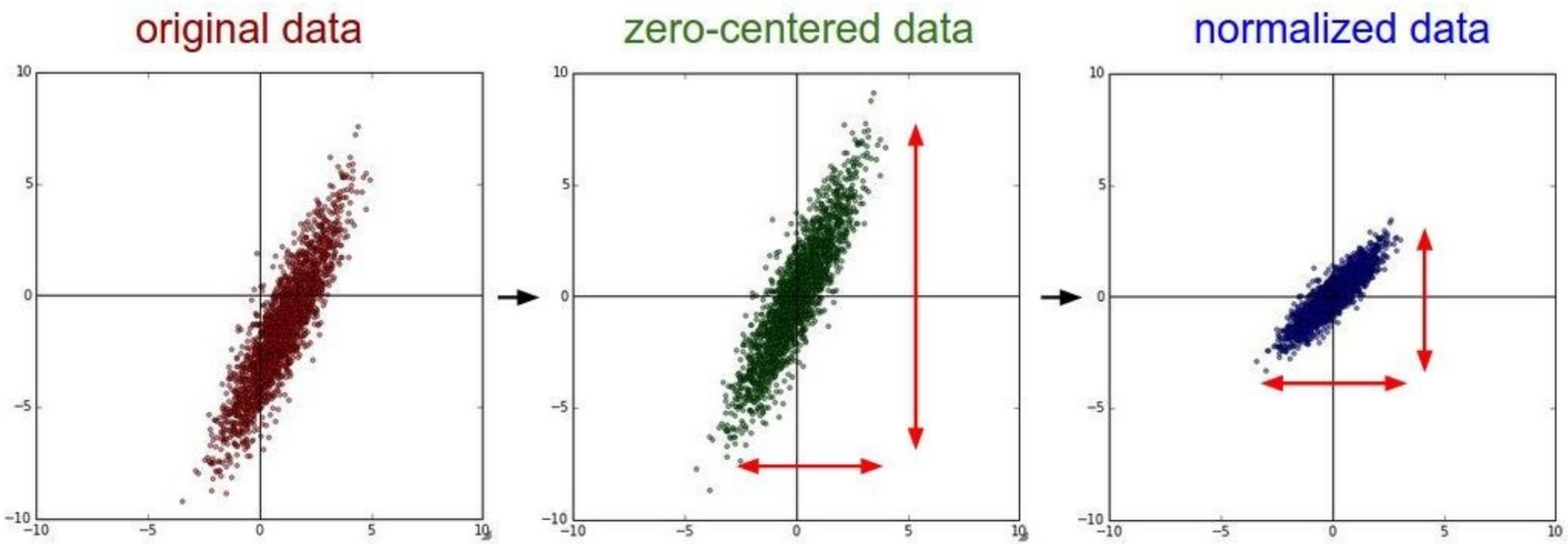
Output the class with the highest score

Recent Trends

- ConvNets stack CONV,POOL,FC layers
- Trend towards smaller filters and deeper architectures
- Trend towards getting rid of POOL/FC layers (just CONV)
- Historically architectures looked like
  - [(CONV-RELU)*N-POOL?]*M-(FC-RELU)*K,SOFTMAX
  - where N is usually up to ~5, M is large, 0 <= K <= 2.
  - but recent advances such as ResNet/GoogLeNet have challenged this paradigm

Normalization

Training CNNs




```
X -= np.mean(X, axis = 0)
```

```
X /= np.std(X, axis = 0)
```

Example Normalization Layer: LayerNorm

- High-level Idea: Learn parameters that let us scale / shift the input data
- Normalize input data
- Scale / shift using learned parameters
- Suppose $x \in \mathbb{R}^{N \times D}$ as input feature/feature maps
- Statistics calculated per batch: $\mu, \sigma \in \mathbb{R}^{N \times 1}$
- Learned parameters applied to each sample: $\gamma, \beta \in \mathbb{R}^{1 \times D}$
- Layer norm: $y = \frac{\gamma(x-\mu)}{\sigma} + \beta$

Batch Normalization

- Want zero-mean unit-variance activations?
- Consider a batch of activations at some layer. To make each dimension zero-mean unit-variance, apply Batch Normalization

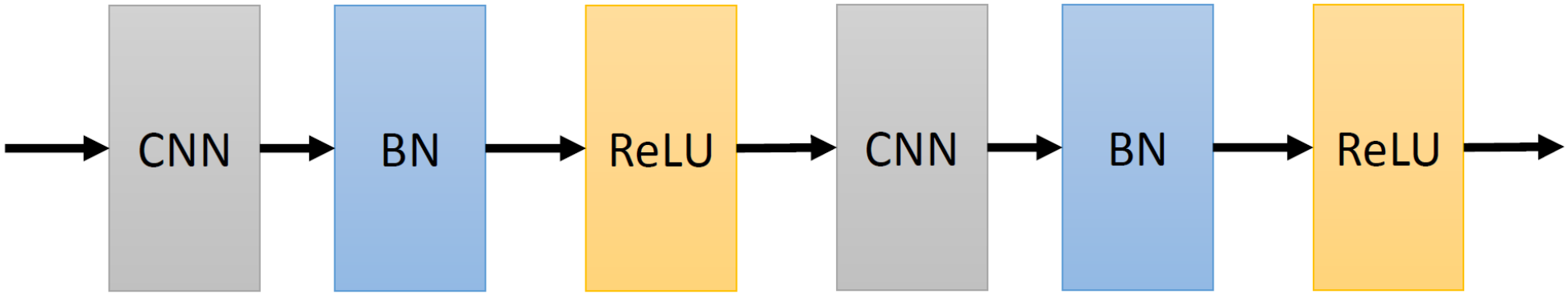
$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$

- Use mini-batch to approximate E and Var
- What if it's wrong?
  - Make sure the network can "correct" this change

$$y^{(k)} = \gamma^{(k)}\widehat{x}^{(k)} + \beta^{(k)}.$$

  - 2 learnable parameters for each x

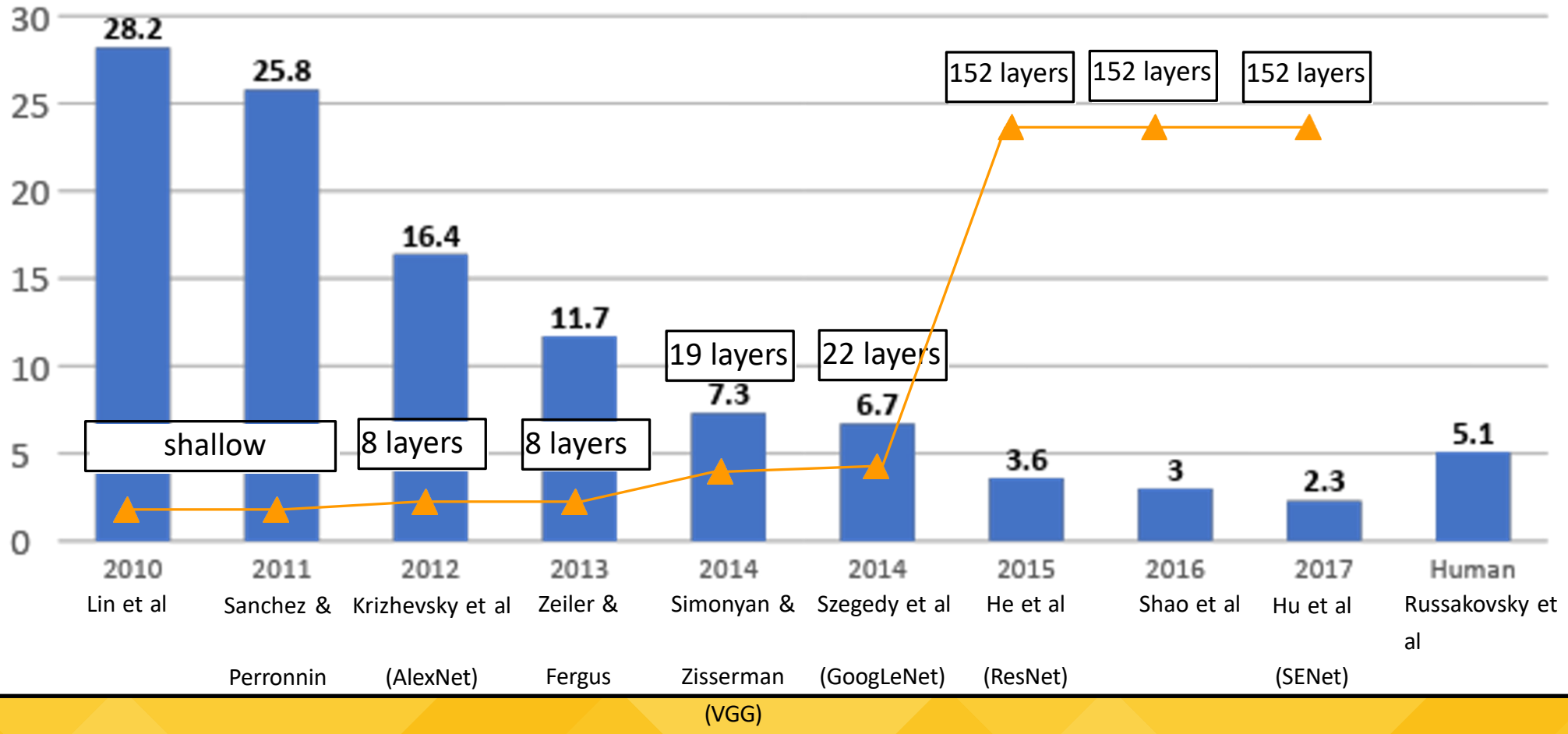
Batch Normalization



- Usually inserted after Convolutional or Fully Connected layers, and before nonlinearity.
- Makes deep networks much easier to train.
- Acts as regularization during training.
- Zero overhead at test-time: can be fused with conv!

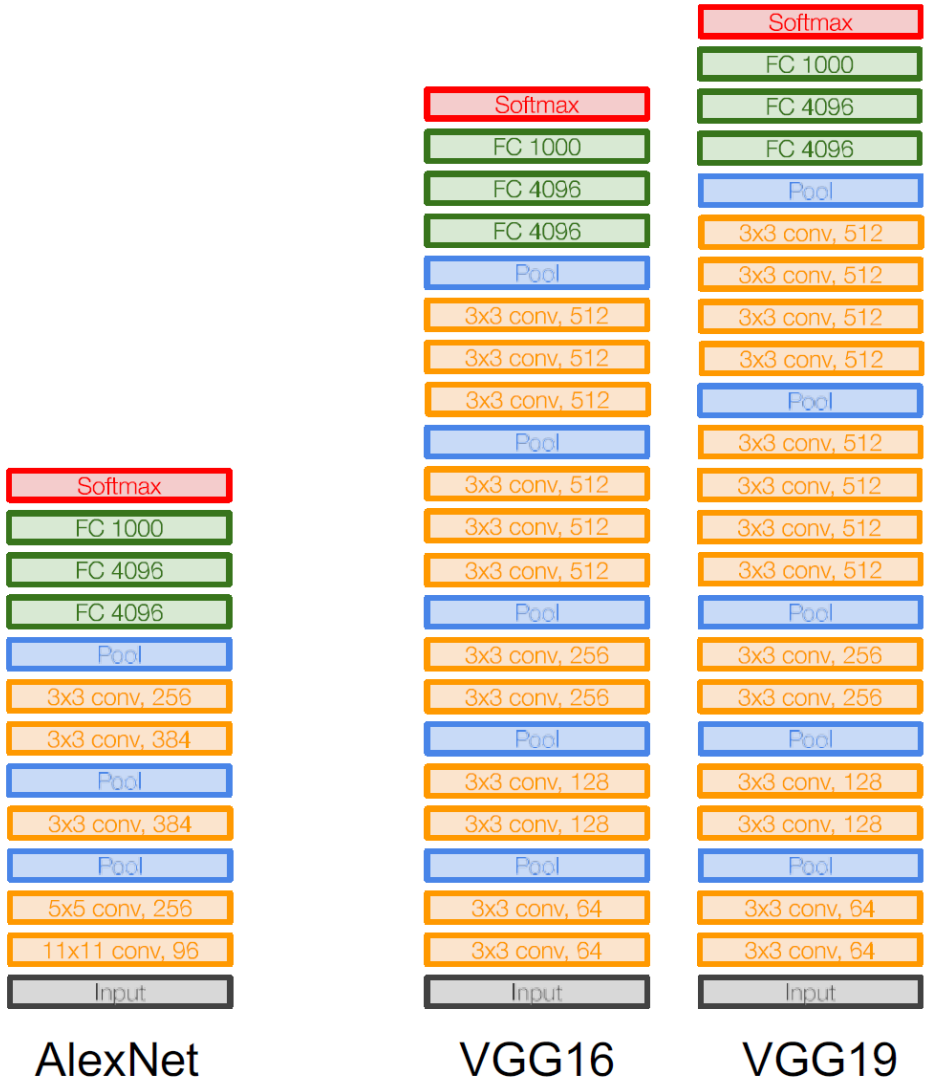
CNN Architectures

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



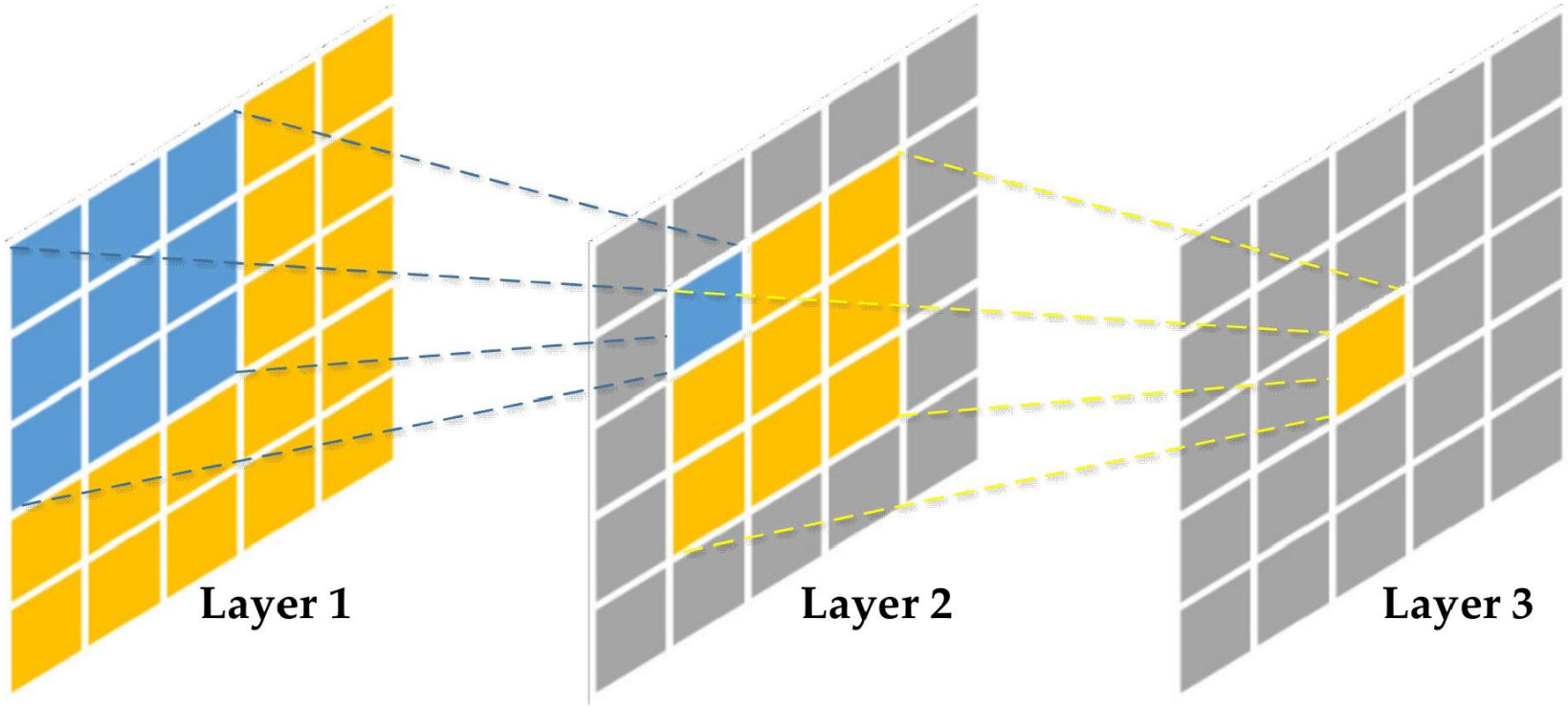
Case Study: VGGNet

- Small filters, Deeper networks
- 8 layers (AlexNet)-> 16 - 19 layers (VGGNet)
- Only 3x3 CONV stride 1, pad 1 and 2x2 MAX POOL stride 2
- 11.7% top 5 error in ILSVRC’13 (ZFNet)
- -> 7.3% top 5 error in ILSVRC’14



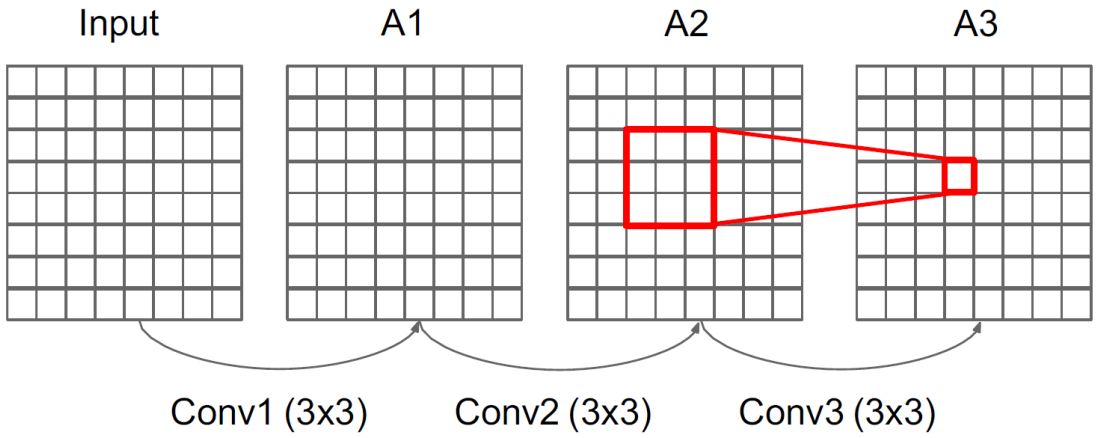
Receptive Field

- Def: The receptive field in CNN refers to the region of the input space that a particular layer's feature is looking at



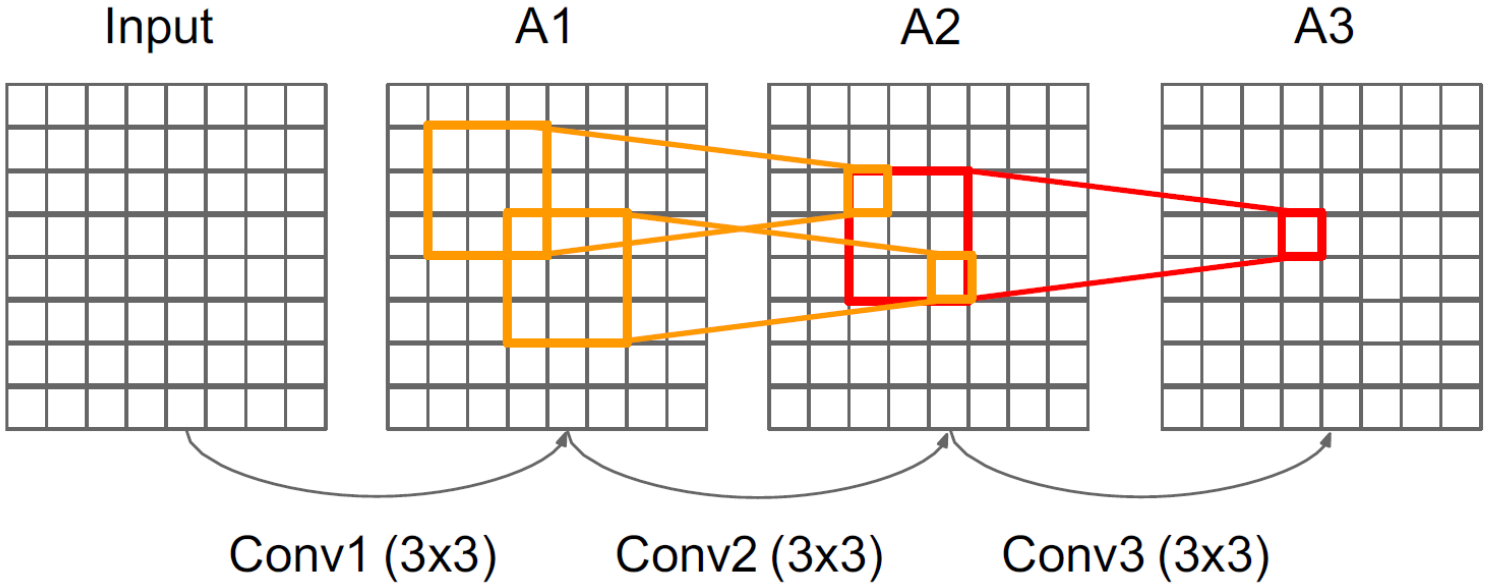
Case Study: VGGNet

- Q: Why use smaller filters? (3x3 conv)
- A: Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer
- Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



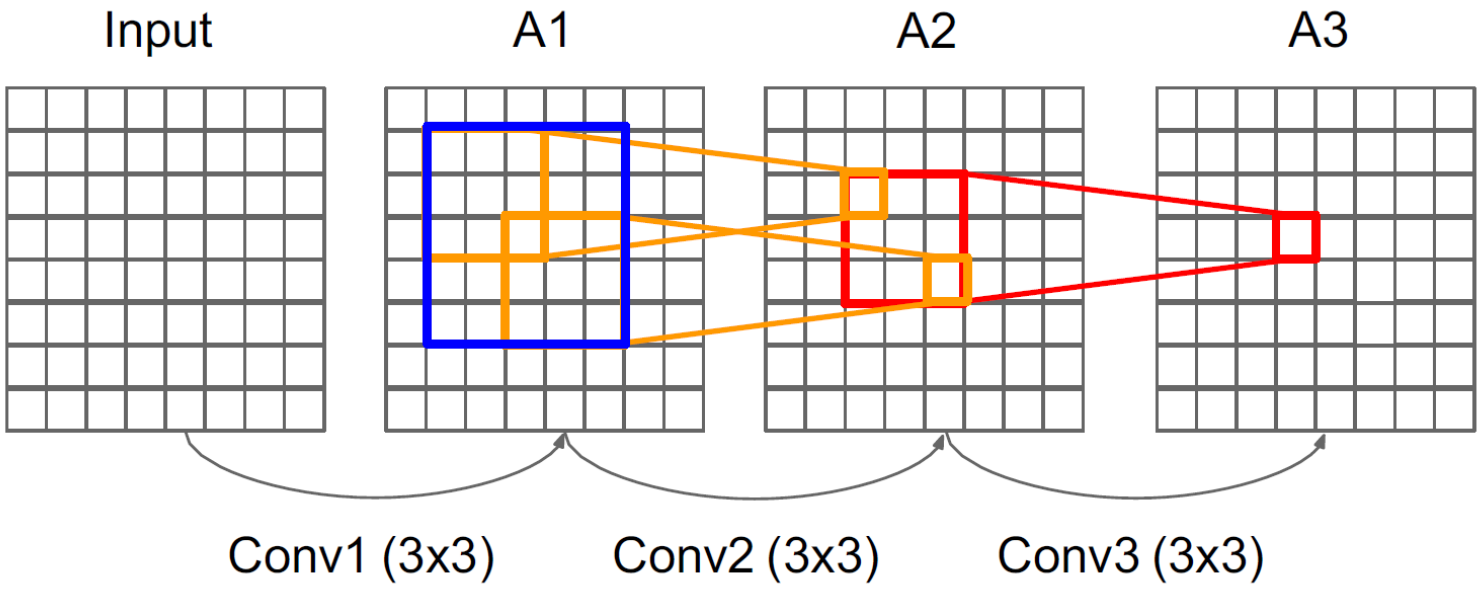
Case Study: VGGNet

- Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



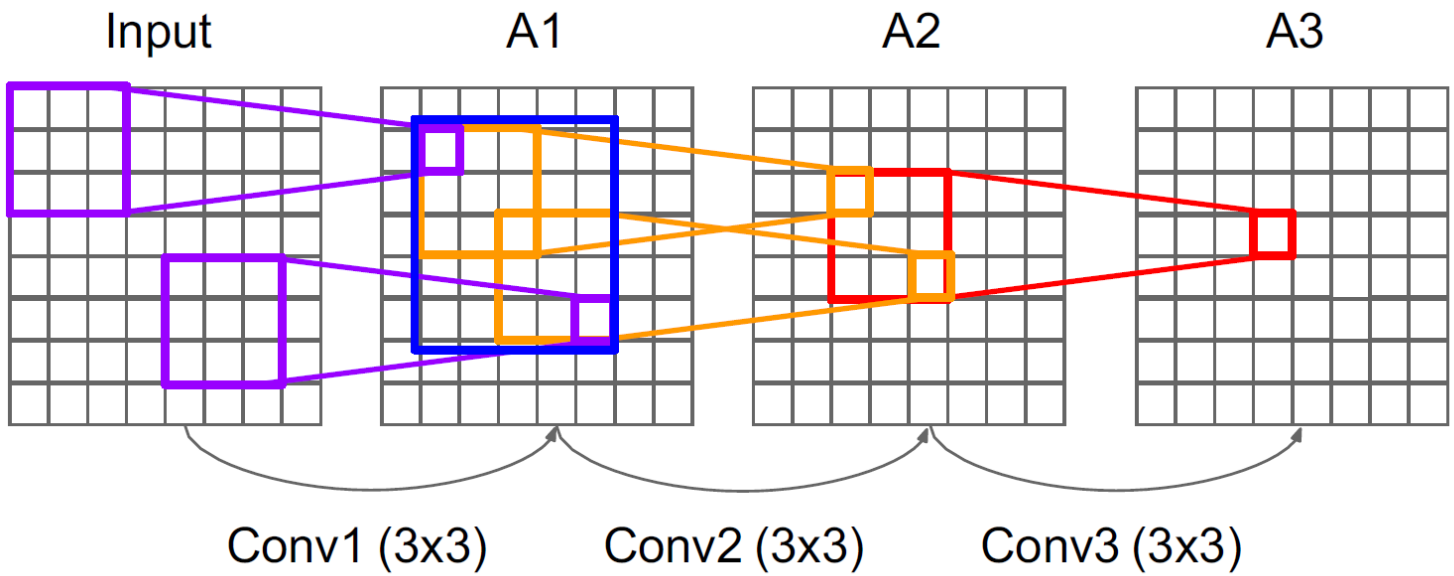
Case Study: VGGNet

- Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



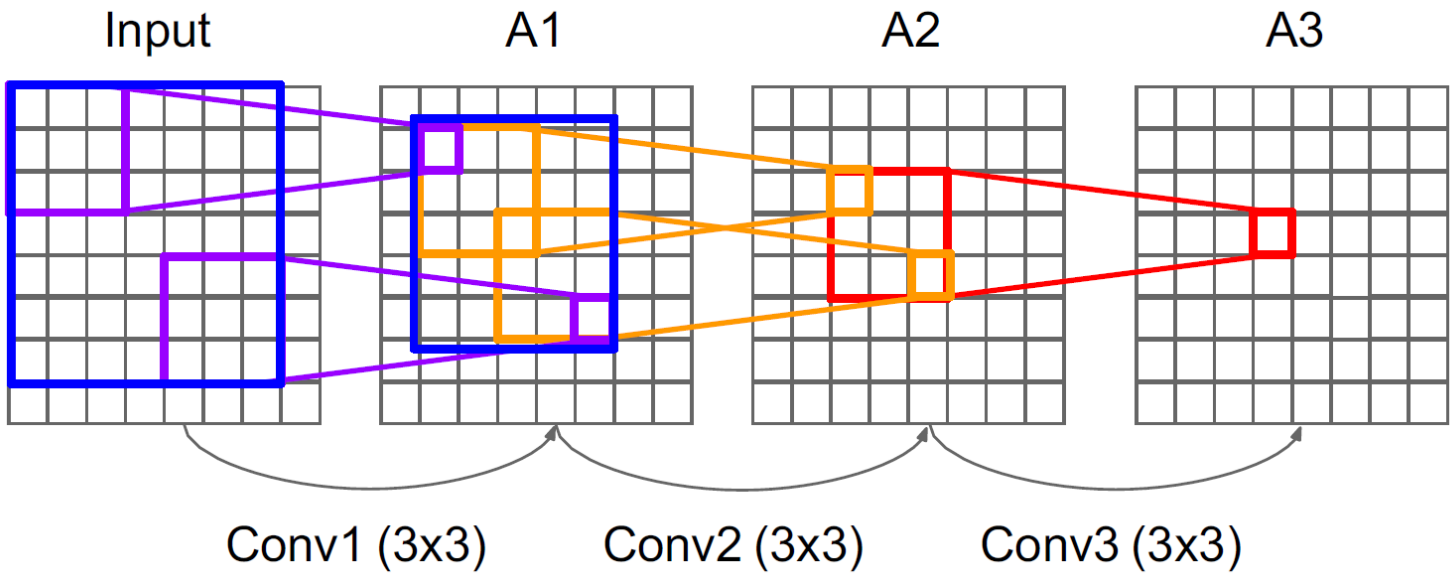
Case Study: VGGNet

- Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



Case Study: VGGNet

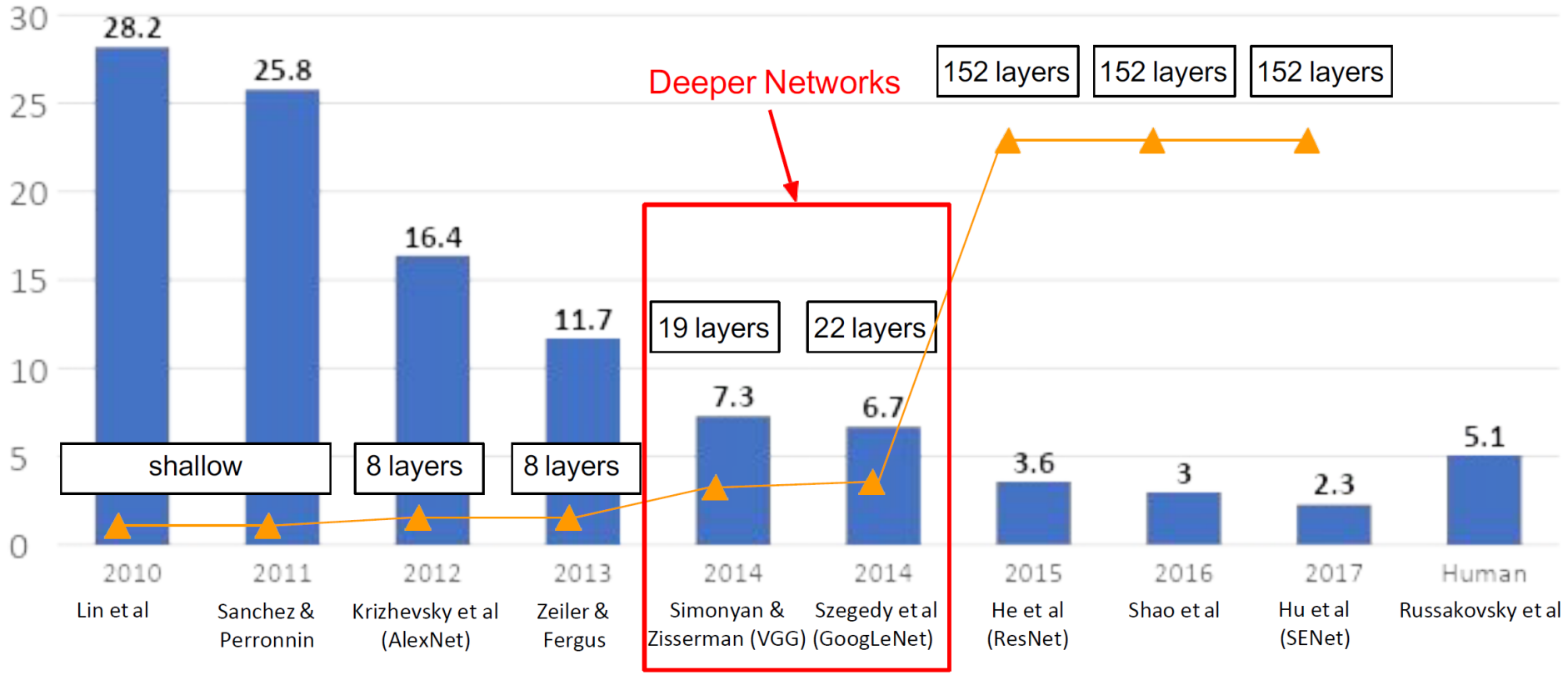
- Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



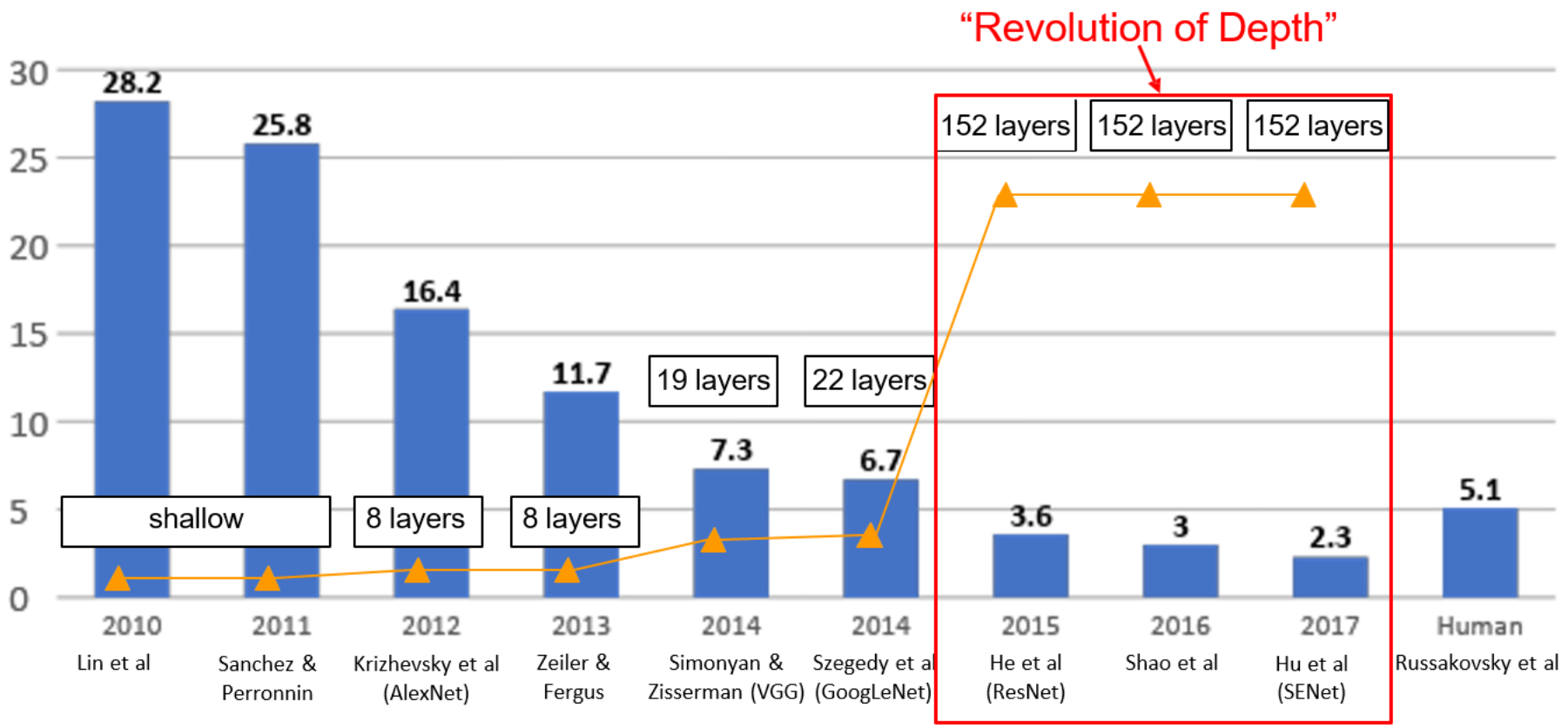
Case Study: VGGNet

- Q: Why use smaller filters? (3x3 conv)
- Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer
- But deeper, more non-linearities
- And fewer parameters: $3 * (3^2C^2)$ vs. 7^2C^2 for C channels per layer

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

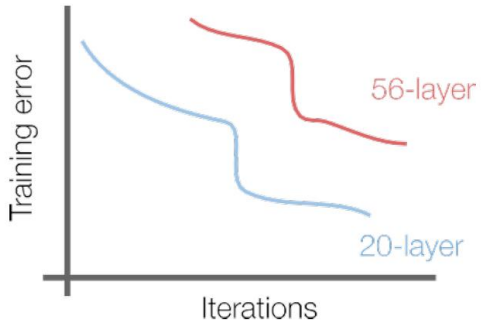


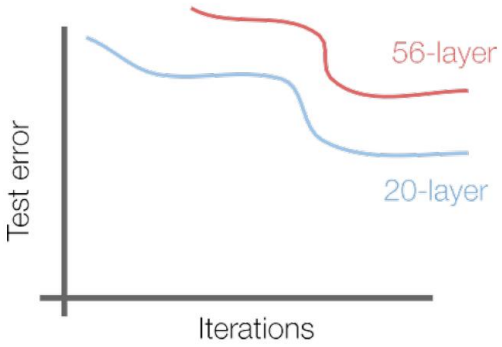
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Case Study: ResNet

- What happens when we continue stacking deeper layers on a “plain” convolutional neural network?





- 56-layer model performs worse on both training and test error
- The deeper model performs worse, but it’s not caused by overfitting

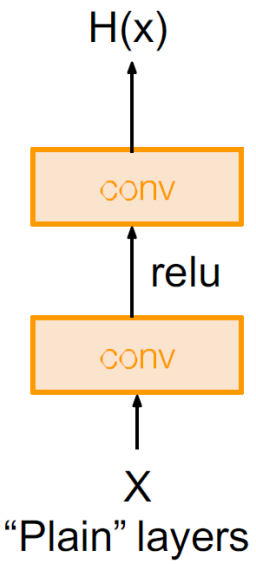
Case Study: ResNet

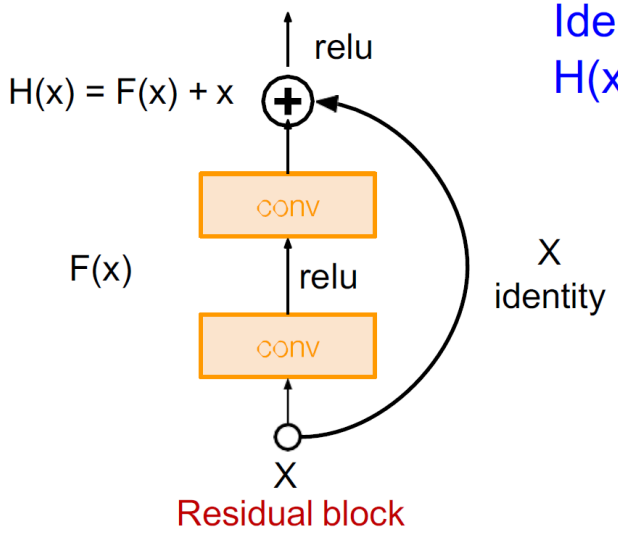
- Fact: Deep models have more representation power (more parameters) than shallower models.
- Hypothesis: the problem is an optimization problem, deeper models are harder to optimize
- What should the deeper model learn to be at least as good as the shallower model?
- A solution by construction is copying the learned layers from the shallower model and setting additional layers to identity mapping.

Case Study: ResNet

- Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

H(x) = F(x) + x





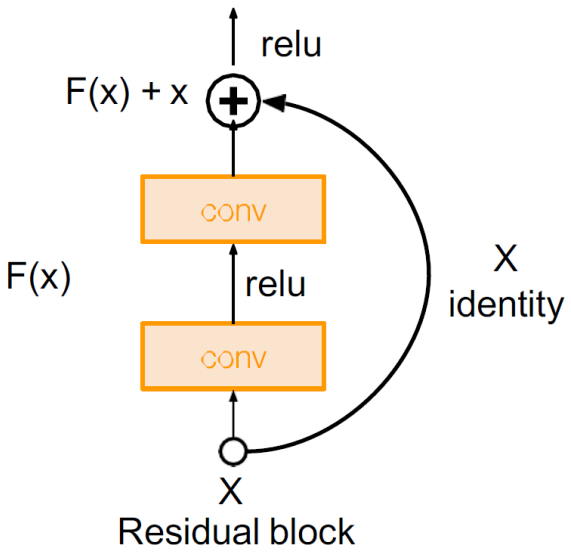
Identity mapping:
H(x) = x if F(x) = 0

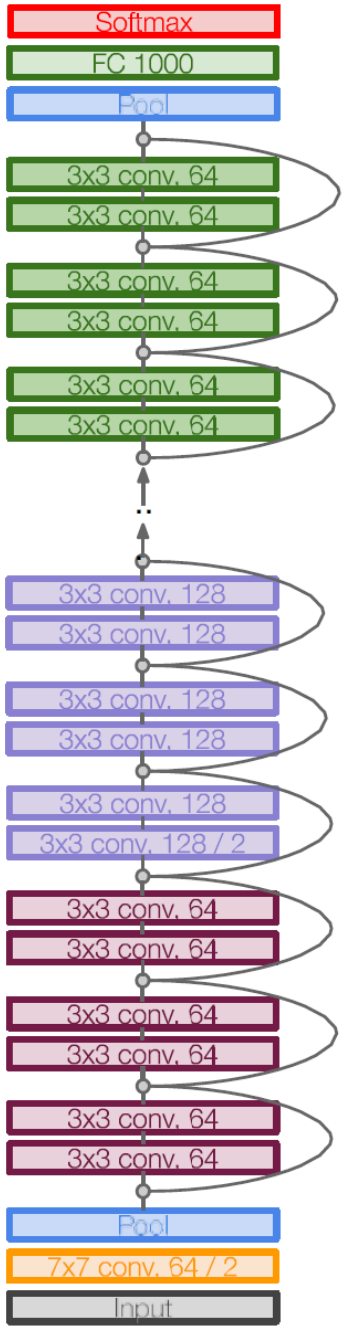
Use layers to fit **residual**
F(x) = H(x) - x

instead of H(x) directly

Case Study: ResNet

- Very deep networks using residual connections
- 152-layer model for ImageNet
- ILSVRC'15 classification winner (3.57% top 5 error)
- Swept all classification and detection competitions in
- ILSVRC'15 and COCO'15!



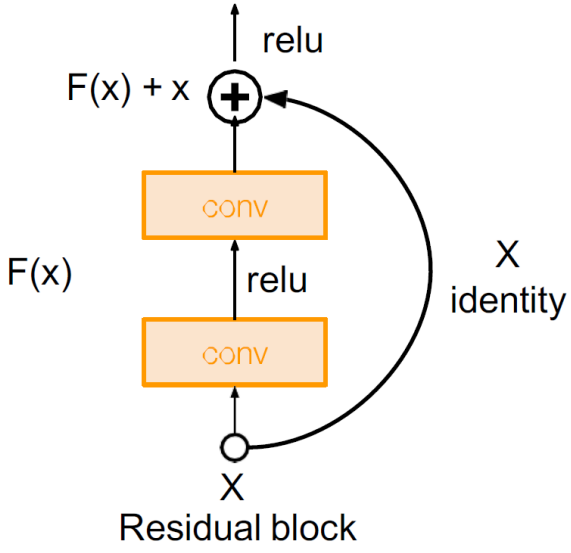


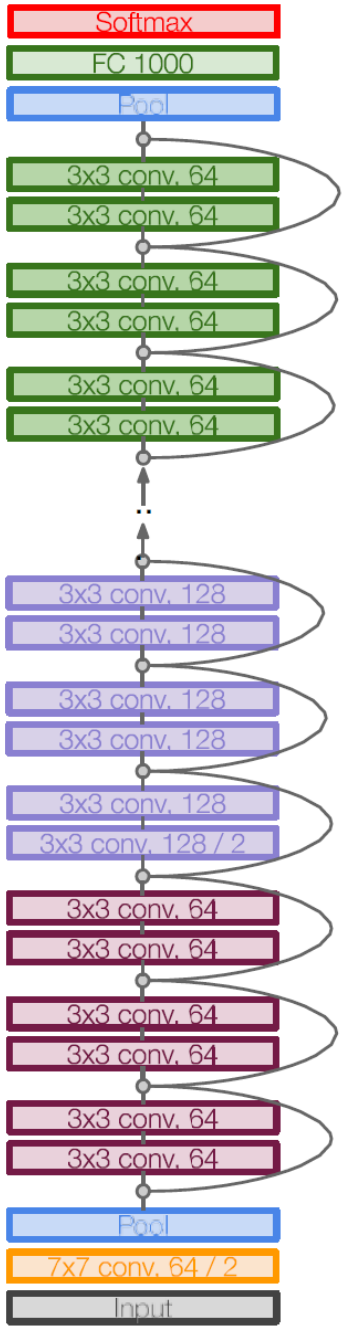
Case Study: ResNet

- Fact: Deep models have more representation power (more parameters) than shallower models.
- Hypothesis: the problem is an optimization problem, deeper models are harder to optimize
- What should the deeper model learn to be at least as good as the shallower model?
- A solution by construction is copying the learned layers from the shallower model and setting additional layers to identity mapping.

Case Study: ResNet

- Full ResNet architecture
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double # of filters and downsample spatially using stride 2 (/2 in each dimension)
- Additional conv layer at the beginning (stem)
- No FC layers at the end (only FC 1000 to output classes)





Case Study: ResNet

- Training ResNet in practice:
  - Batch Normalization after every CONV layer
  - Xavier initialization from He et al.
  - SGD + Momentum (0.9)
  - Learning rate: 0.1, divided by 10 when validation error plateaus
  - Mini-batch size 256
  - Weight decay of 1e-5
  - No dropout used

Weight Initialization

Weight Initialization Case: Values too small

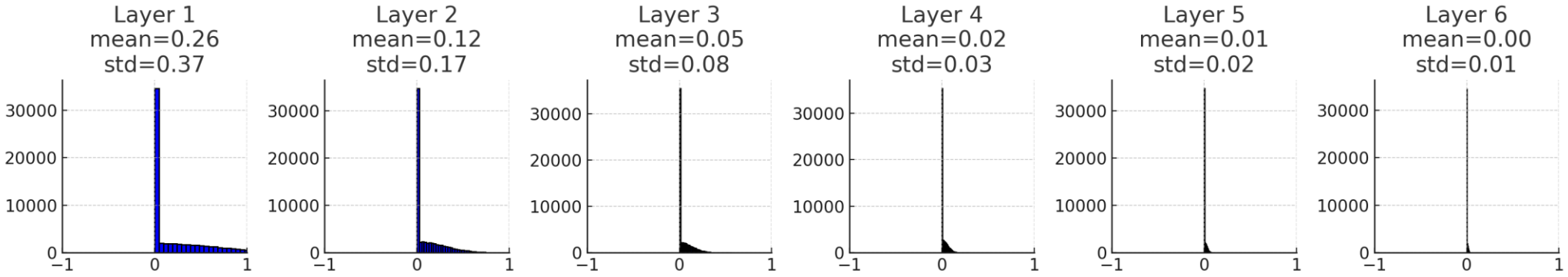
```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)  # Small weight init
    x = np.maximum(0, x.dot(W))  # ReLU activation
    hs.append(x)
```

- Forward pass for a 6-layer net with hidden size 4096

Weight Initialization Case: Values too small

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout)  # Small weight init
    x = np.maximum(0, x.dot(W))  # ReLU activation
    hs.append(x)
```

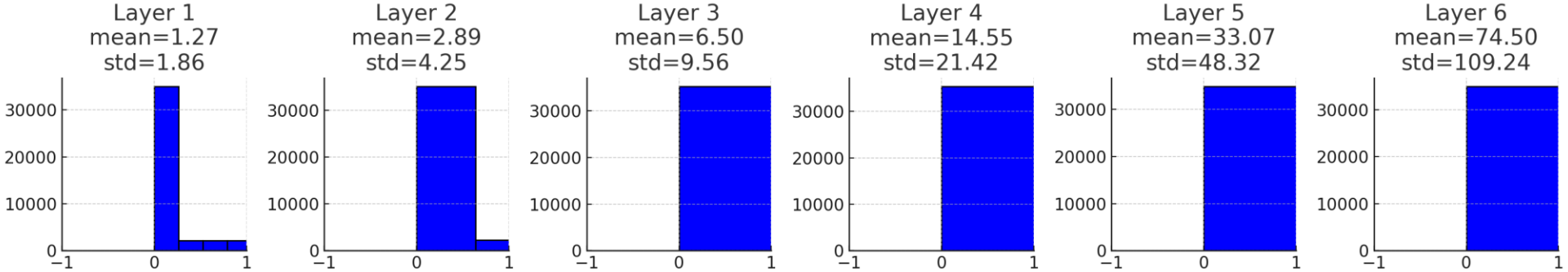
- All activations tend to zero for deeper network layers



Weight Initialization Case: Values too small

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout)  # Small weight init
    x = np.maximum(0, x.dot(W))  # ReLU activation
    hs.append(x)
```

- Increase std of initial weights from 0.01 to 0.05



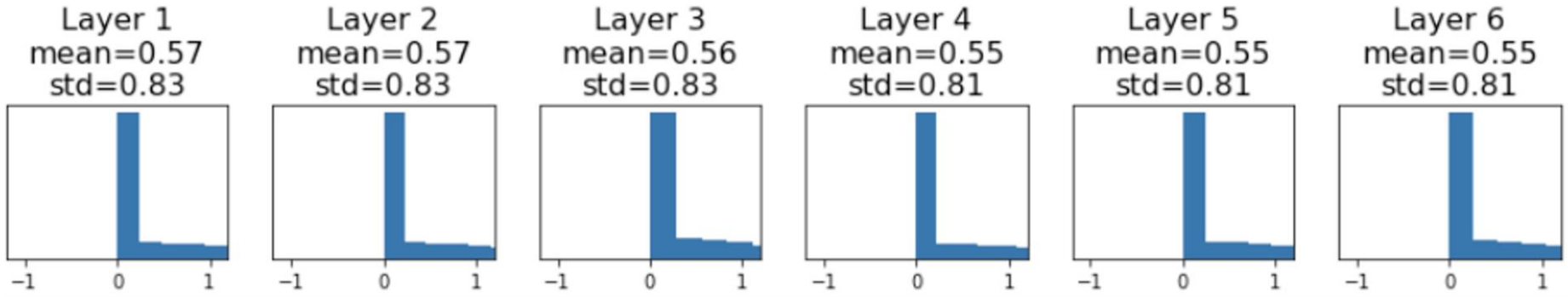
- Activations blow up quickly

How to fix this? Depends on the size of the layer

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

ReLU correction: std = sqrt(2 / Din)

"Just right": Activations are nicely scaled for all layers!



He et al, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification", ICCV 2015

CNN Training – Data Preprocessing & Augmentation

TLDR for Image Normalization: center and scale for each channel

- Subtract per-channel mean and
- Divide by per-channel std (almost all modern models)
  - (stats along each channel = 3 numbers)
- Requires pre-computing means and std for each pixel channel (given your dataset)

```
norm_pixel[i,j,c] = (pixel[i,j,c] - np.mean(pixel[:,:,c])) / np.std(pixel[:,:,c])
```

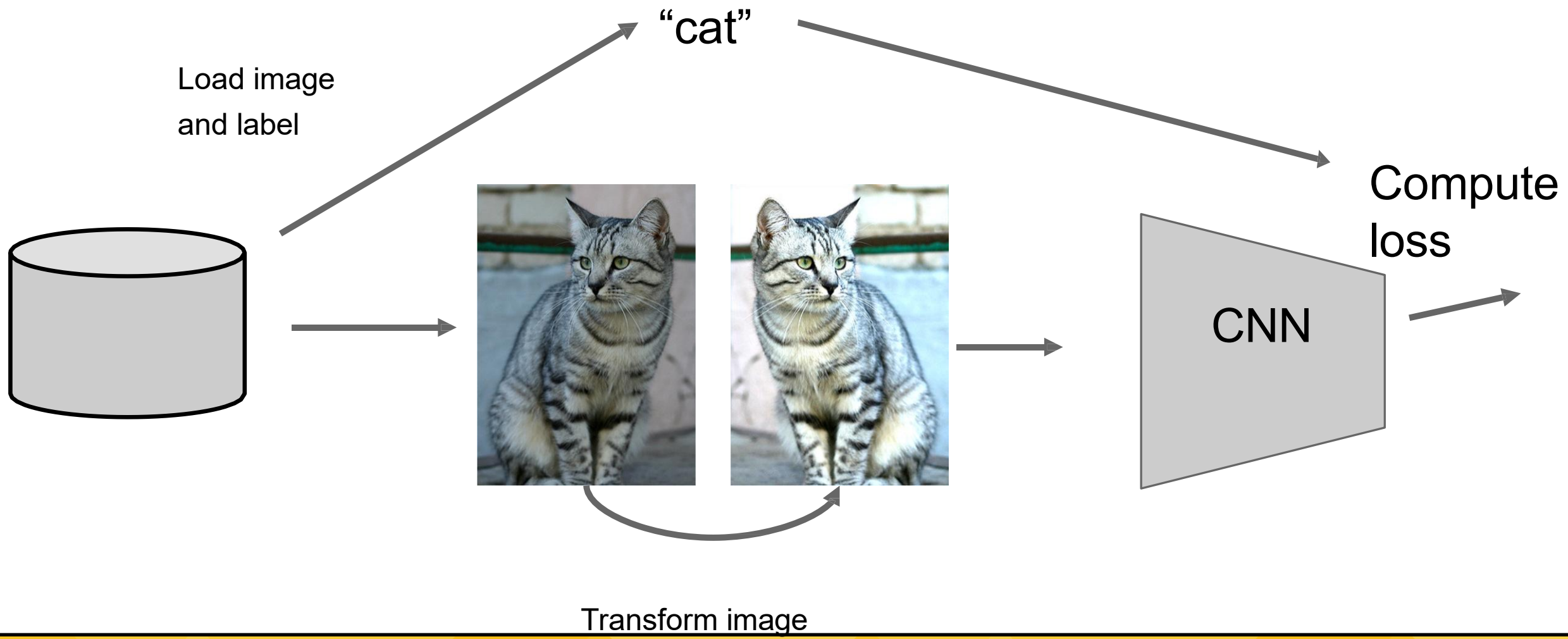
Regularization: A common pattern

- **Training**: Add some kind of randomness: $y = f_w(x, z)$
- **Testing:** Average out randomness (sometimes approximate)

$$y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$$

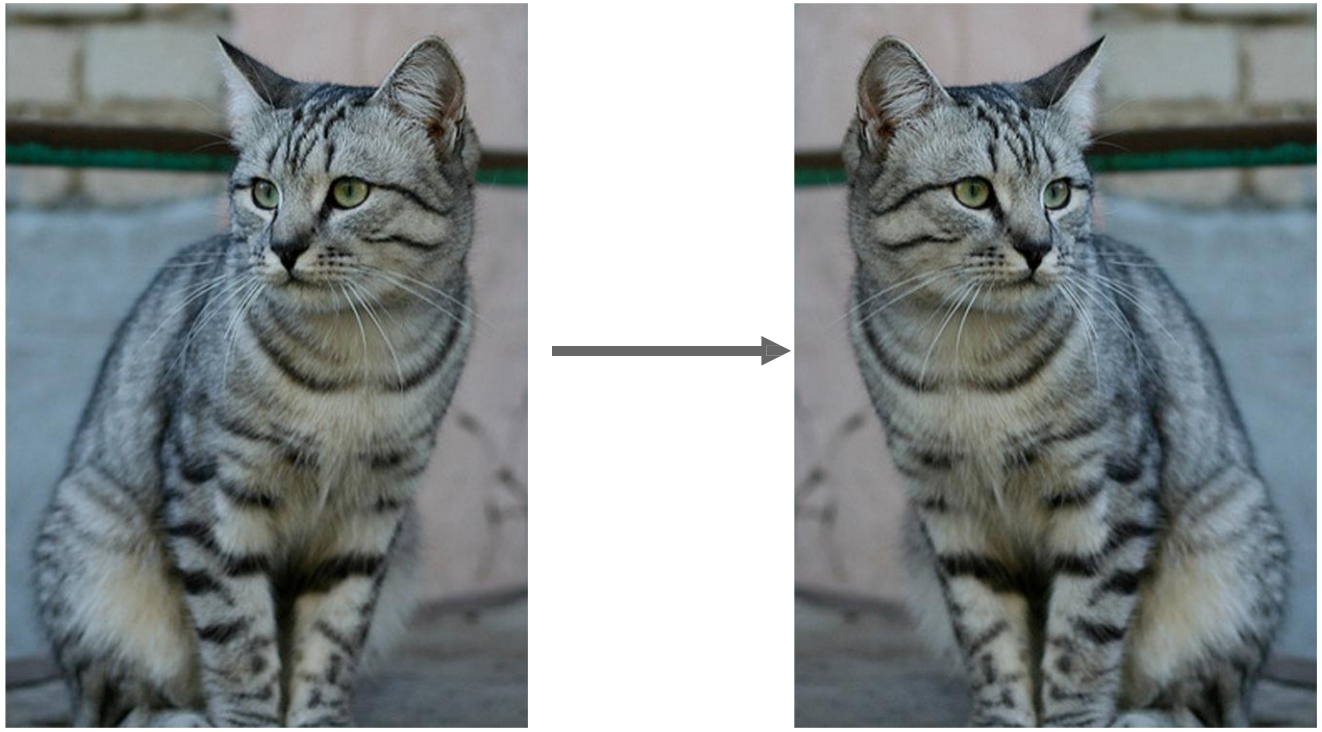
- Example: Dropout
- Training: Randomly drop activations
- Testing: Use all activations and average values with p

Regularization: Data Augmentation



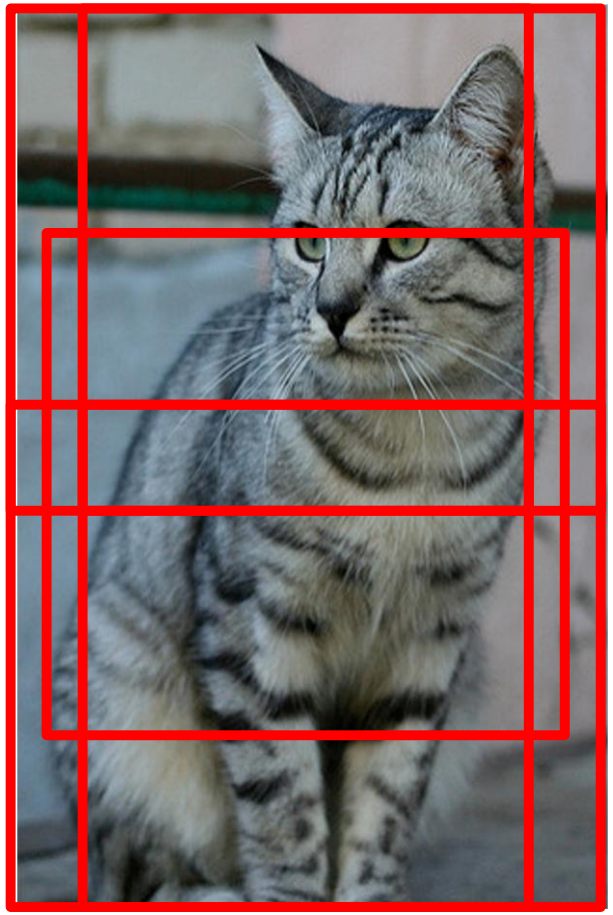
Data Augmentation

- Horizontal Flips



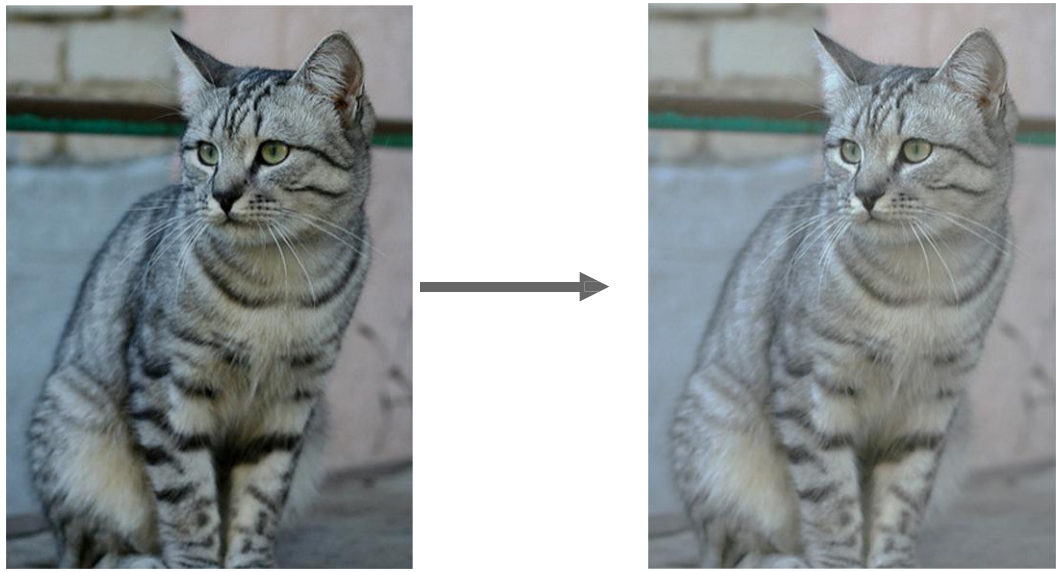
Data Augmentation

- Random crops and scales
- Training: sample random crops / scales
  - Pick random L in range [256, 480]
  - Resize training image, short side = L
  - Sample random 224 x 224 patch
- Test Time Augmentation: average a fixed set of crops
  - Resize image at 5 scales: {224, 256, 384, 480, 640}
  - For each size, use 10 224 x 224 crops: 4 corners + center, + flips



Data Augmentation

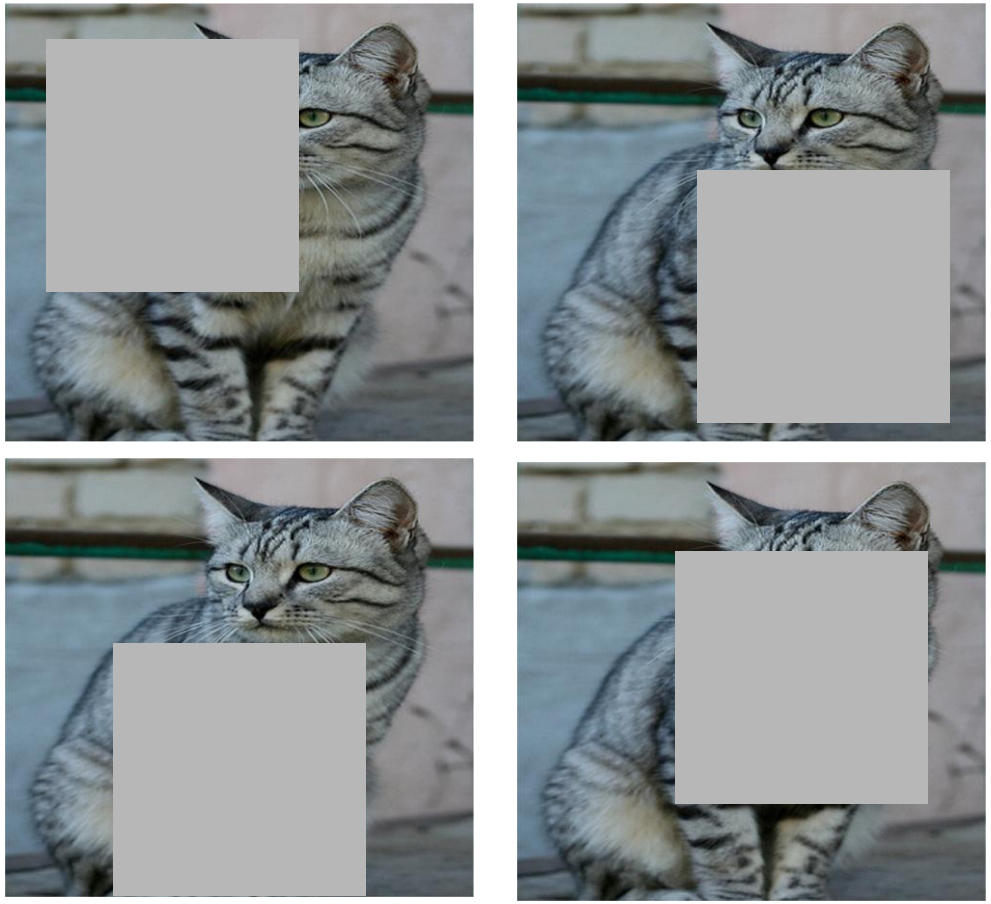
- Color Jitter
- Simple: Randomize contrast and brightness



Regularization: Cutout

- **Training**: Set random image regions to zero
- **Testing**: Use full image

- **Examples**:
- Dropout
- Data Augmentation Cutout / Random Crop

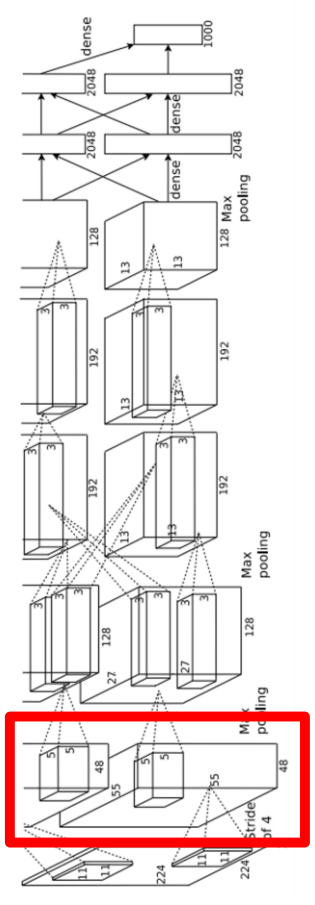


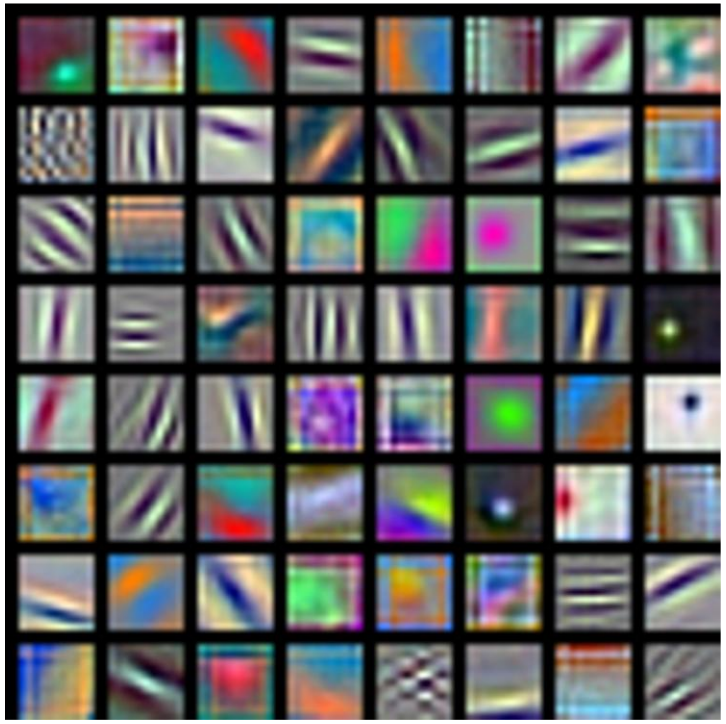
DeVries and Taylor, “Improved Regularization of Convolutional Neural Networks with Cutout”, arXiv 2017

Transfer Learning

- What if you don’t have a lot of data?
- Can you still train CNNs?

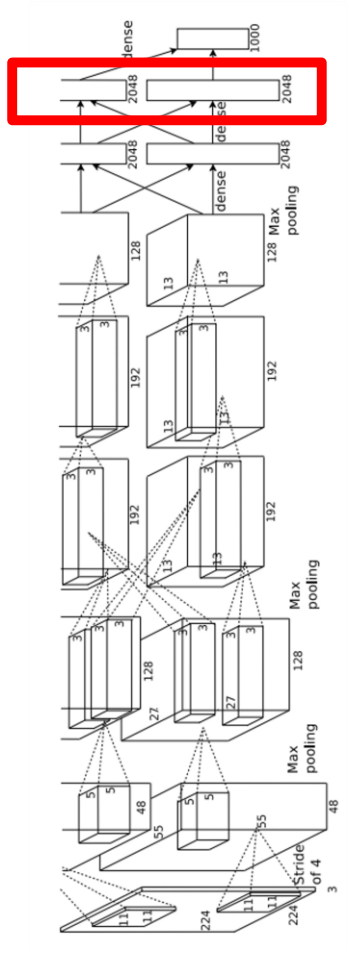
Transfer Learning



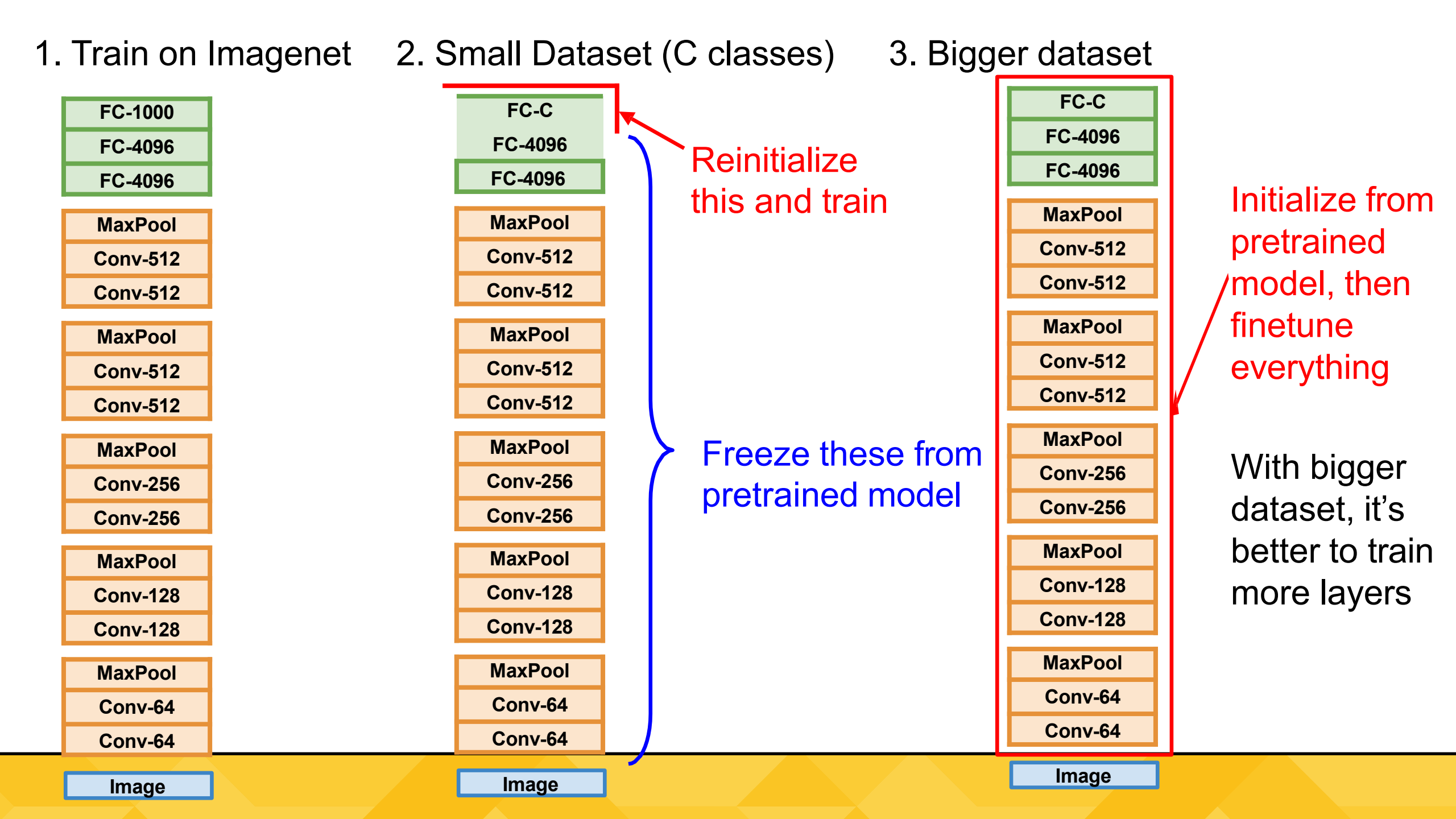


AlexNet:
64 x 3 x 11 x 11

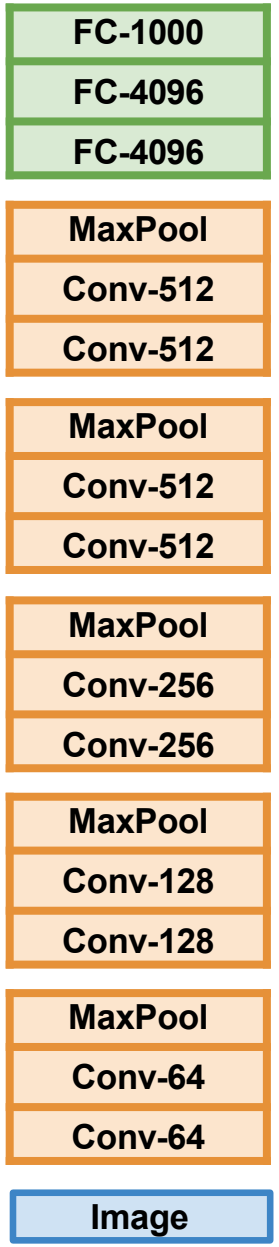
Transfer Learning



Test image L2 Nearest neighbors in <u>feature</u> space


1. Train on Imagenet
2. Small Dataset (C classes)
3. Bigger dataset
FC-1000
FC-4096
FC-4096
MaxPool
Conv-512
Conv-512
MaxPool
Conv-512
Conv-512
MaxPool
Conv-256
Conv-256
MaxPool
Conv-128
Conv-128
MaxPool
Conv-64
Conv-64
Image
FC-C
FC-4096
FC-4096
MaxPool
Conv-512
Conv-512
MaxPool
Conv-512
Conv-512
MaxPool
Conv-256
Conv-256
MaxPool
Conv-128
Conv-128
MaxPool
Conv-64
Conv-64
Image
Reinitialize this and train
Freeze these from pretrained model
FC-C
FC-4096
FC-4096
MaxPool
Conv-512
Conv-512
MaxPool
Conv-512
Conv-512
MaxPool
Conv-256
Conv-256
MaxPool
Conv-128
Conv-128
MaxPool
Conv-64
Conv-64
Image
Initialize from pretrained model, then finetune everything
With bigger dataset, it's better to train more layers

Transfer Learning

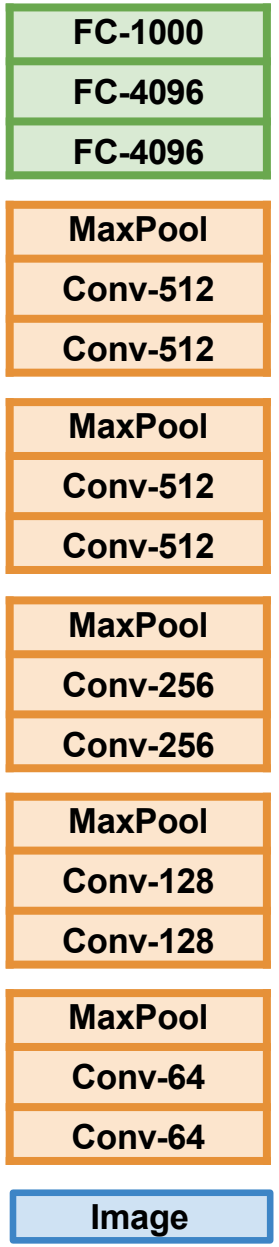


More specific

More generic

	very similar dataset	very different dataset
very little data	?	?
quite a lot of data	?	?

Transfer Learning

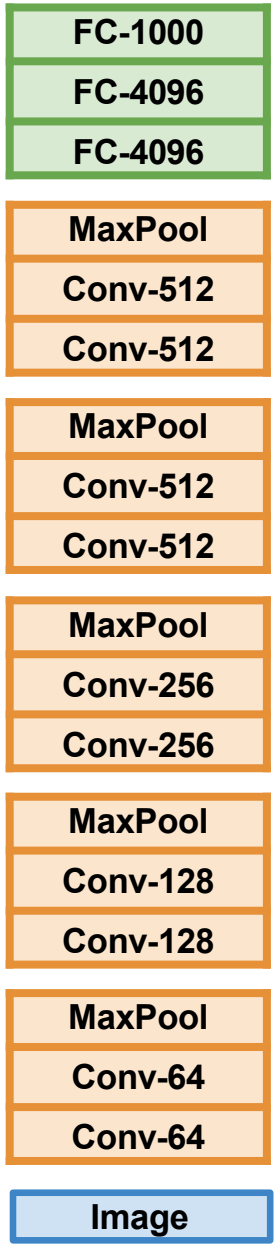


More specific

More generic

	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on final layer	?
quite a lot of data	Finetune all model layers	?

Transfer Learning



More specific

More generic

	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on final layer	Try another model or collect more data
quite a lot of data	Finetune all model layers	Either finetune all model layers or train from scratch!

Takeaway for your projects and beyond

- Have some dataset of interest but it has < ~1M images?
- Find a very large dataset that has similar data, train a big model there
- Transfer learn to your dataset
- Deep learning frameworks provide a “Model Zoo” of pretrained models so you don’t need to train your own
- PyTorch: TorchVision
- Huggingface: HuggingFaceHuggingFace

Choosing Hyperparameters

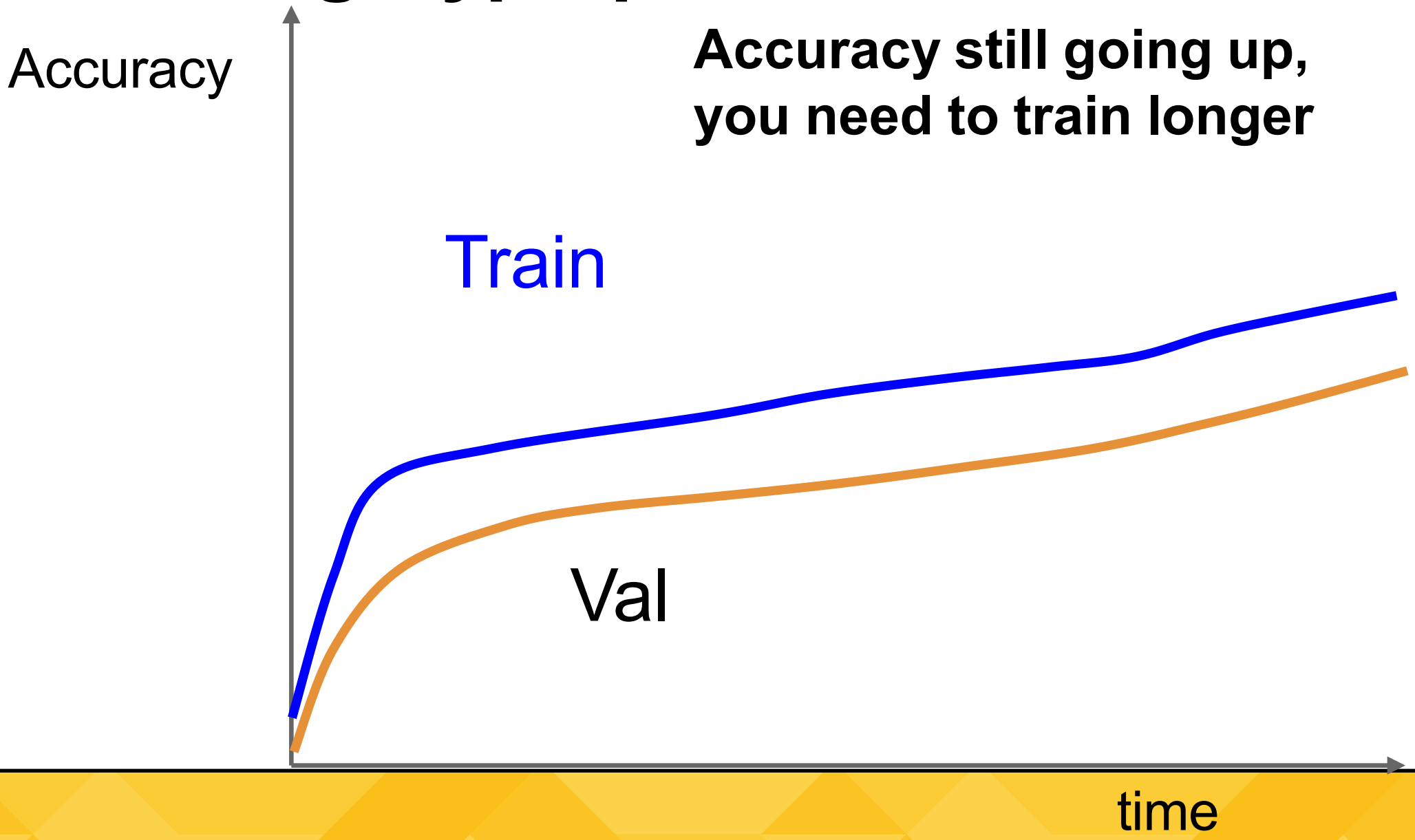
- Step 1: Check initial loss
- Step 2: Overfit a small sample
- Step 3: Find LR that makes loss go down

- Use the architecture from the previous step, use all training data, turn on small weight decay, find a learning rate that makes the loss drop significantly within ~100 iterations
- Good learning rates to try: 1e-1, 1e-2, 1e-3, 1e-4, 1e-5

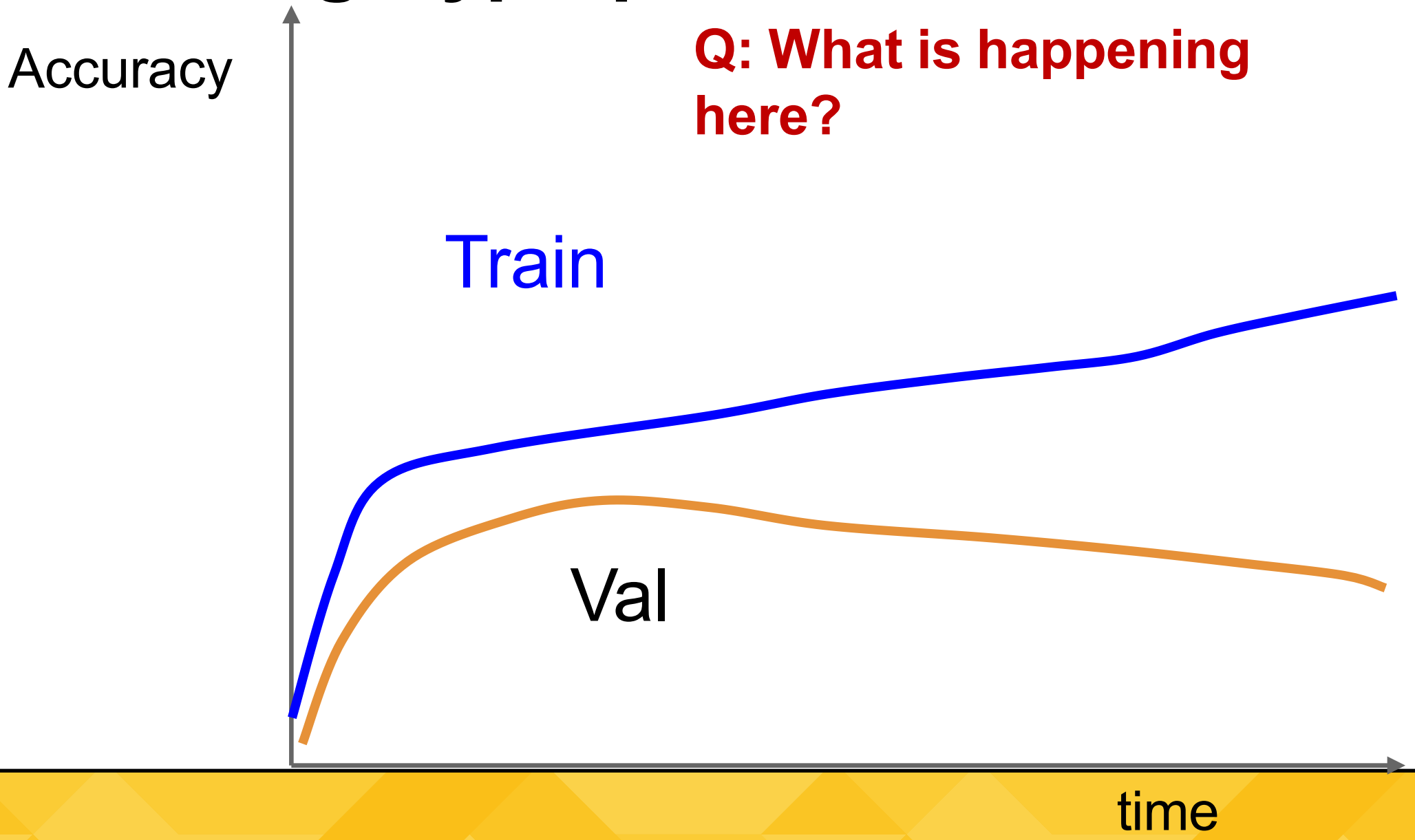
Choosing Hyperparameters

- Step 1: Check initial loss
- Step 2: Overfit a small sample
- Step 3: Find LR that makes loss go down
- Step 4: Coarse grid of hyperparams, train for ~1-5 epochs
- Step 5: Refine grid, train longer
- Step 6: Look at loss and accuracy curves (next slides)

Choosing Hyperparameters

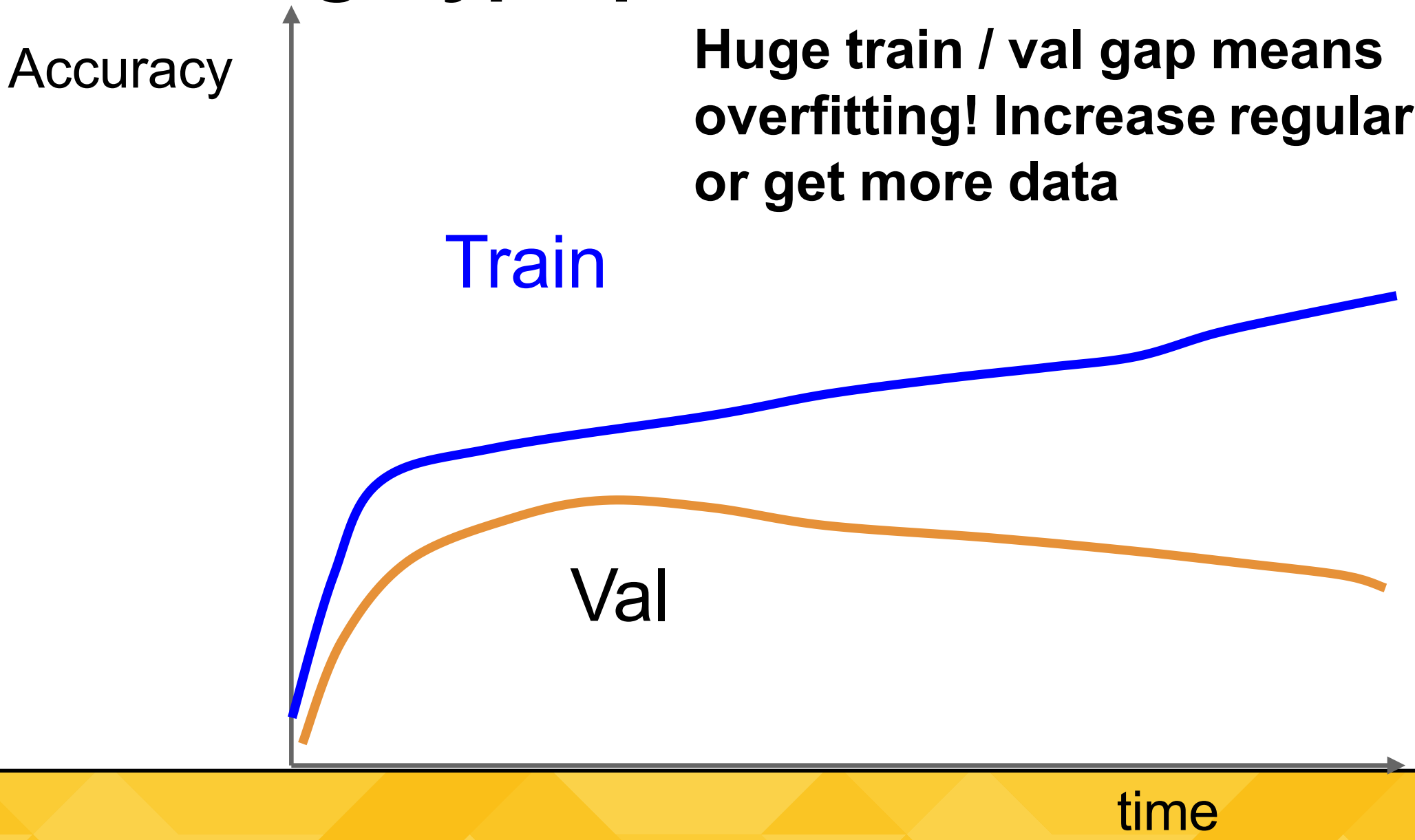


Choosing Hyperparameters



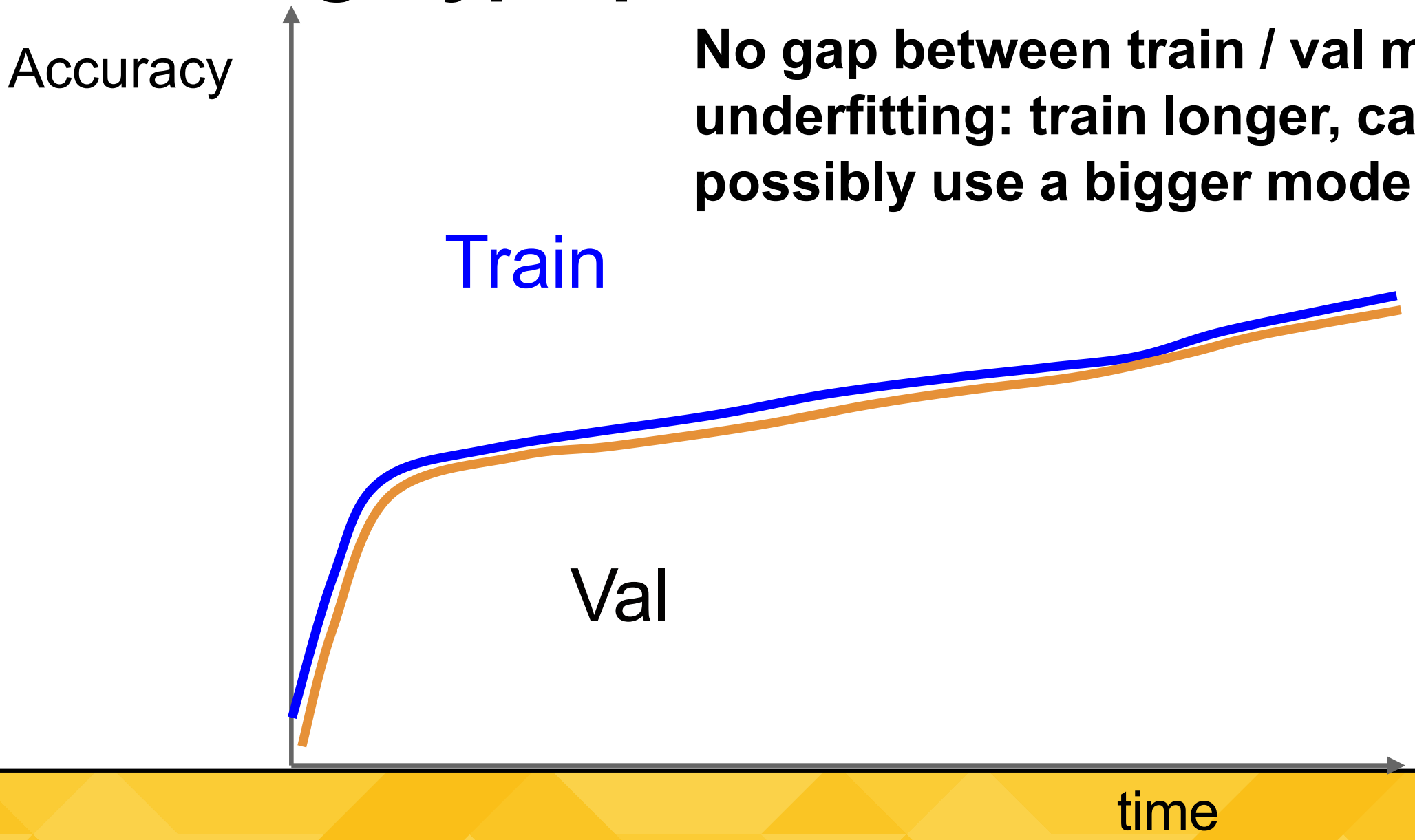
Choosing Hyperparameters

Huge train / val gap means overfitting! Increase regularization or get more data



Choosing Hyperparameters

No gap between train / val means underfitting: train longer, can possibly use a bigger model



Choosing Hyperparameters

- Step 1: Check initial loss
- Step 2: Overfit a small sample
- Step 3: Find LR that makes loss go down Step 4: Coarse grid, train for ~1-5 epochs Step 5: Refine grid, train longer
- Step 6: Look at loss and accuracy curves
- Step 7: GOTO step 5

Random Search vs. Grid Search

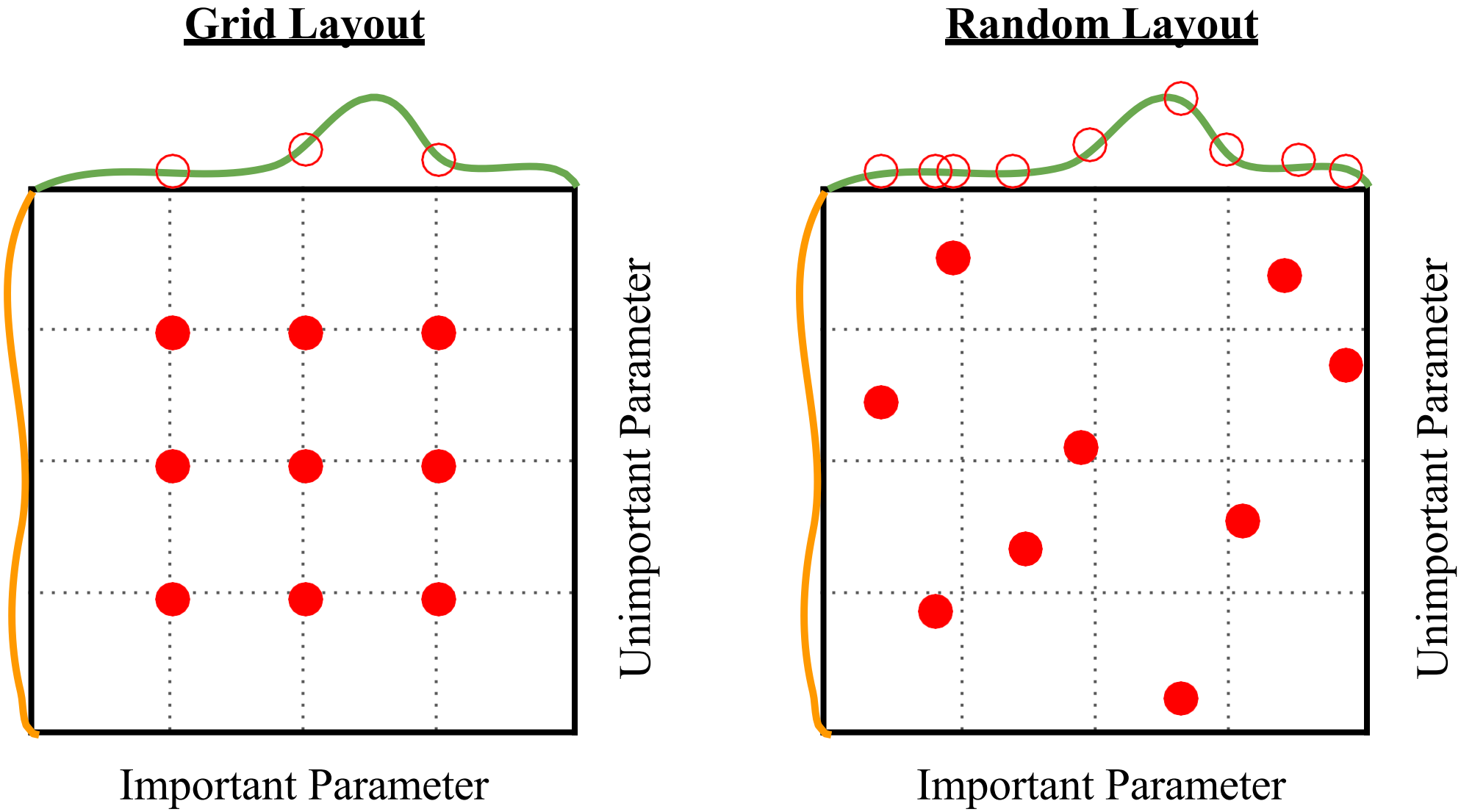


Illustration of Bergstra et al., 2012 by Shayne Longpre, copyright CS231n 2017

More

- **AlexNet** showed that you can use CNNs to train Computer Vision models.
- **VGG** shows that deeper networks work better.
- **ResNet** showed us how to train extremely deep networks
  - Limited only by GPU & memory!
  - Showed diminishing returns as networks got bigger
- After ResNet: CNNs were better than the human metric and focus shifted to Efficient networks:
  - Lots of tiny networks aimed at mobile devices: **MobileNet**, **ShuffleNet**
  - Neural Architecture Search (NAS) can now automate architecture design

Summary

- Applications of convolutional neural networks
- Convolution/Pooling layer
- Techniques in CNN
  - Data preprocessing, Batch normalization, Early stopping, Regularization, Dropout
- CNN architectures
  - AlexNet, VGG, ResNet

References

- Stanford CS231n